

Pattern: Modular Monolith

Problem: A simple monolith becomes a tightly coupled "big ball of mud"; full microservices add operational cost a small team can't absorb.

Solution: Single deployable unit, but internal code split into strict, well-bounded modules (business domains) with enforced interfaces and a shared database with schema-level isolation.

Use when: Small/medium teams, unstable domain boundaries, strong consistency needed, you want microservices-like modularity without distribution.

Avoid when: You need independent per-component deploy/scale, multiple teams shipping at different cadences, or strong failure isolation.

Trade-offs: Gain in-process speed, simple deploy/debug, transactional consistency, lower infra cost; pay with no independent scaling and risk of boundary erosion without enforcement tooling/discipline.

Related: Microservices, Layered, Loose Coupling.

Source: martinfowler.com (MonolithFirst); Shopify Engineering (Deconstructing the Monolith).

Pattern: Microservices

Problem: A large monolith blocks independent deployment, scaling, and team autonomy.

Solution: Decompose into independently deployable, loosely coupled services, each owning one business capability and its own data store, communicating over the network. Services are typically sized to a "two-pizza team."

Use when: Large orgs, distinct scaling profiles per component, need for independent deploy and tech heterogeneity, well-understood boundaries.

Avoid when: Small teams, early/unstable domain, simple apps - Fowler's "microservice premium" outweighs the benefit.

Trade-offs: Gain independent deploy/scale, fault isolation, team autonomy; pay with distributed-system complexity (network latency, partial failure, eventual consistency, ops/observability overhead).

Related: Modular Monolith, API Gateway, Database per Service, Saga.

Source: Fowler & Lewis, Microservices (2014) and Microservice Premium / Microservice Trade-Offs; microservices.io; AWS Microservices on AWS whitepaper.

Pattern: Layered / N-Tier Architecture

Problem: Mixing UI, business logic, and data access produces unmaintainable, untestable code.

Solution: Organize into horizontal layers (presentation, business, persistence/data), each depending only on the layer below; layers are logical, tiers are physical.

Use when: Traditional enterprise/CRUD apps, complex business rules with straightforward scaling, lift-and-shift migrations.

Avoid when: You need independent scaling of features or very low latency; strict layering can become a bottleneck.

Trade-offs: Gain separation of concerns, testability, swap-a-layer flexibility; pay with cross-layer call overhead and the "sinkhole" anti-pattern (layers passing through with no logic), plus coupling if boundaries are violated.

Related: Modular Monolith, Loose Coupling.

Source: Azure Architecture Center (Architecture Styles - N-tier); martinfowler.com (PresentationDomainDataLayering).

Pattern: Synchronous Request/Response (REST over HTTP)

Problem: A client needs an immediate, correlated answer from a service.

Solution: Client sends an HTTP request (typically RESTful, JSON) and blocks until the service responds; stateless, cacheable, uniform interface.

Use when: Low-latency reads, CRUD, user-facing queries needing an immediate result, simple integrations.

Avoid when: Long-running work, spiky loads, or when the caller shouldn't block; chained sync calls create latency and "locked" code prone to timeouts.

Trade-offs: Gain simplicity, immediate result, easy debugging; pay with temporal coupling (both sides up simultaneously), cascading failure risk, and reduced resilience under load.

Related: API Gateway, Async Messaging, Circuit Breaker, Retry.

Source: AWS Microservices on AWS (Communication mechanisms); martinfowler.com; Azure Architecture Center.

Pattern: Asynchronous Messaging / Queue-Based Load Leveling

Problem: A fast/spiky producer overwhelms a slower consumer; direct calls couple them and drop work under load.

Solution: Place a durable queue between producer and consumer; producer enqueues and moves on, consumer pulls at its own rate. One consumer processes each message (competing consumers to scale out).

Use when: Variable/bursty load, tasks decomposable into async units, you need to decouple intake from processing throughput.

Avoid when: Caller needs a low-latency synchronous reply, or volume is low and stable (queue adds needless complexity).

Trade-offs: Gain resilience, load smoothing, independent scaling; pay with eventual consistency, one-way comms (need a separate reply path), and queue-depth/latency growth if consumers can't keep up.

Related: Pub/Sub, Competing Consumers, Event-Driven, Circuit Breaker.

Source: Azure Architecture Center (Queue-Based Load Leveling; Competing Consumers); AWS Microservices on AWS; AWS Serverless Lens.

Pattern: Event-Driven Architecture / Publisher-Subscriber

Problem: A sender must notify many interested consumers without knowing who they are, and direct coupling blocks independent evolution.

Solution: Publishers emit events to a topic/broker; multiple subscribers each receive a copy and react independently (fan-out). Senders don't know receivers.

Use when: Broadcast/fan-out, reactive workflows, decoupling teams/services, integrating SaaS/cross-account.

Avoid when: You need a single atomic transaction across publisher and consumers, or strict immediate consistency/ordering.

Trade-offs: Gain loose coupling, scalability, isolated failures, easy add/remove of consumers; pay with eventual consistency, no global view of behavior (hard to reason/debug), and delivery concerns (duplicates, poison messages, ordering).

Related: Async Messaging, CQRS, Event Sourcing, Saga.

Source: martinfowler.com (The Many Meanings of Event-Driven Architecture, 2017); Azure Architecture Center (Publisher-Subscriber); AWS Serverless Lens.

Pattern: API Gateway

Problem: Clients calling dozens of microservices directly face chatty calls, coupling, and duplicated cross-cutting concerns.

Solution: Single entry point that routes/proxies requests, aggregates responses, and centralizes auth, TLS termination, rate limiting, and caching. Variant: Backends-for-Frontends (per client type).

Use when: Non-trivial microservices with multiple client types; you want to centralize cross-cutting concerns.

Avoid when: Simple apps with one service/client; gateway adds overhead and a chokepoint.

Trade-offs: Gain a clean unified interface, fewer round-trips, centralized security; pay with a potential single point of failure/bottleneck, an extra network hop, and a component that can over-couple to internal services if bloated.

Related: Microservices, BFF, Loose Coupling, Gateway Offloading.

Source: microservices.io (API Gateway / BFF); Azure Architecture Center; AWS Microservices on AWS.

Pattern: Cache-Aside (Lazy Loading)

Problem: Repeatedly reading the same data from a slow/expensive store hurts performance and scale.

Solution: App checks cache first; on miss, loads from the store, populates the cache, and returns. On write, update store and invalidate cache; use TTL/expiration tuned to access pattern.

Use when: Read-heavy workloads, data tolerant of slight staleness, hot keys expensive to fetch.

Avoid when: Write-heavy or strongly-consistent data, or when most requests miss (cache overhead exceeds benefit).

Trade-offs: Gain lower latency, higher throughput, reduced store load; pay with cache-store inconsistency (stale reads), added complexity (invalidation/expiry tuning), and cold-cache miss penalties.

Related: CQRS, Materialized View, Horizontal Scaling.

Source: Azure Architecture Center (Cache-Aside); AWS ElastiCache / Reliable Web App guidance.

Pattern: Command Query Responsibility Segregation (CQRS)

Problem: One model for reads and writes becomes complex and can't tune/scale reads and writes independently.

Solution: Separate the write model (commands, validation, business rules) from the read model (denormalized DTOs/views), optionally with separate data stores kept in sync via events. Coined by Greg Young (~2010), extending Bertrand Meyer's Command-Query Separation.

Use when: High read:write asymmetry, complex domains, task-based UIs, reads needing independent scaling/tuning.

Avoid when: Simple CRUD, models nearly identical, or the team lacks experience with eventually-consistent systems - Fowler warns CQRS adds risky complexity for most systems.

Trade-offs: Gain independent read/write optimization and scaling, security separation, clarity; pay with significant complexity, eventual consistency / read-model lag (stale reads), and doubled storage + sync infra.

Related: Event Sourcing, Event-Driven, Database per Service, API Composition.

Source: martinfowler.com/bliki/CQRS.html (Greg Young); Azure Architecture Center (CQRS).

Pattern: Database per Service

Problem: Sharing one database across services creates tight coupling and blocks independent deployment/scaling.

Solution: Each service owns its private database/schema; other services access the data only via the owning service's API, never directly.

Use when: Microservices needing loose coupling, independent schema evolution, and polyglot persistence per service's needs.

Avoid when: Strong cross-entity consistency/transactions are required, or the system is small enough that a shared DB suffices.

Trade-offs: Gain loose coupling, independent deploy/scale, best-fit storage per service; pay with no cross-service ACID transactions (need Saga), harder cross-service queries (need API Composition/CQRS), and operational multiplicity.

Related: Microservices, Saga, API Composition, CQRS.

Source: microservices.io (Database per Service; Shared Database anti-pattern); AWS Microservices on AWS.

Pattern: Saga (Distributed Transaction)

Problem: A business transaction spans multiple services, each with its own DB, so a single ACID transaction is impossible.

Solution: Implement as a sequence of local transactions; each step publishes an event/message triggering the next, and failures trigger compensating transactions that undo prior steps. Orchestration or choreography.

Use when: Cross-service transactions in a database-per-service setup where eventual consistency is acceptable.

Avoid when: An ACID transaction in one service suffices, or the domain can't tolerate intermediate/eventually-consistent states.

Trade-offs: Gain consistency without distributed locks and better availability/scalability; pay with high implementation complexity (compensation logic, state management), eventual consistency, and harder debugging/testing.

Related: Database per Service, Event-Driven, Transaction Outbox, CQRS.

Source: Garcia-Molina & Salem, Sagas, ACM SIGMOD 1987 (doi:10.1145/38713.38742); microservices.io (Saga); AWS Microservices on AWS (Step Functions).

Pattern: Data/Model Separation (AI)

Problem: Baking knowledge or data into model weights (or app code) makes updates require costly retraining/redeploys and obscures governance.

Solution: Keep the model (reasoning/behavior) separate from the data/knowledge it uses; supply knowledge at runtime via retrieval, feature stores, or external data stores rather than embedding it.

Use when: Knowledge changes often, needs source-backing/auditability, or must be governed separately from model logic.

Avoid when: Knowledge is static and small, or behavior (not knowledge) is the thing that must change - then fine-tuning fits.

Trade-offs: Gain update agility without retraining, clearer governance, reuse of one model across data sets; pay with runtime retrieval latency/complexity and dependence on retrieval quality.

Related: RAG, Cache-Aside, Loose Coupling, Online/Batch Inference.

Source: AWS Prescriptive Guidance (RAG vs fine-tuning); martinowler.com (Emerging Patterns in Building GenAI Products).

Pattern: Horizontal Scaling with Stateless Services

Problem: Vertical scaling hits a single-machine ceiling and creates a single point of failure; server-side session state ties users to specific nodes.

Solution: Make services stateless (offload session/state to external store/cache/DB), then scale out by adding identical instances; any instance can serve any request.

Use when: Variable/growing traffic, web/app tiers, cloud-native designs needing elasticity and fault tolerance.

Avoid when: Inherently stateful workloads (some databases/legacy) where distribution overhead is high - vertical scaling may fit better.

Trade-offs: Gain near-limitless elastic scale, resilience to node failure, easy instance replacement; pay with externalized-state complexity/latency and the need for load balancing and distributed-system discipline.

Related: Load Balancing, Cache-Aside, Microservices, Auto Scaling.

Source: AWS Well-Architected (REL05-BP06 Make systems stateless); AWS Serverless Lens (Share nothing); AWS Architecture Blog.

Pattern: Load Balancing

Problem: Concentrating traffic on one instance overloads it and risks total outage.

Solution: Distribute incoming requests across multiple healthy instances (and across availability zones) via a load balancer, with health checks to route around failures.

Use when: Any horizontally scaled, stateless tier needing high availability and even traffic distribution.

Avoid when: Single-instance workloads or where stickiness/affinity requirements undermine even distribution.

Trade-offs: Gain high availability, fault tolerance, smooth scaling; pay with added infra to manage and the need for genuinely stateless/health-checkable instances.

Related: Horizontal Scaling, Circuit Breaker, API Gateway.

Source: AWS Well-Architected (Reliability Pillar); AWS Architecture Blog (Architecting for Reliable Scalability).

Pattern: Circuit Breaker

Problem: Repeatedly calling a failing dependency wastes resources, blocks threads, and cascades failures system-wide.

Solution: A proxy tracks recent failures; when a threshold is crossed it "opens" and fails fast for a cooldown, periodically allowing trial ("half-open") requests to test recovery before "closing." Popularized by Michael Nygard in Release It! (2007).

Use when: Remote calls to dependencies that may fail for a prolonged time; pair with Retry for transient faults.

Avoid when: Faults are purely transient (use Retry alone), or platform/mesh already handles failure isolation.

Trade-offs: Gain fast failure, protection from cascading outages, time for the dependency to recover; pay with added state/config (thresholds, timeouts) and the need to handle the open state gracefully (fallbacks/degraded mode).

Related: Retry, Bulkhead, Load Balancing, Health Endpoint Monitoring.

Source: martinfowler.com/bliki/CircuitBreaker.html (Nygard, Release It!); Azure Architecture Center (Circuit Breaker); AWS Reliable Web App.

Pattern: Retry with Exponential Backoff

Problem: Transient faults (brief network/service blips) cause requests to fail even though a retry would succeed.

Solution: Transparently retry failed operations with increasing delays (backoff), ideally with jitter and a capped attempt count; only retry idempotent operations.

Use when: Short-lived transient faults on remote calls; the operation is safe to repeat.

Avoid when: Faults are long-lasting (use Circuit Breaker) or the operation is non-idempotent (risk double effects, e.g., double charge).

Trade-offs: Gain resilience to transient faults with minimal code (often built into SDKs); pay with added latency, risk of duplicate side effects, and "retry storms" amplifying load if backoff/jitter is absent.

Related: Circuit Breaker, Idempotency, Async Messaging.

Source: Azure Architecture Center (Retry); AWS Serverless Lens (idempotency / design for failures & duplicates).

Pattern: Retrieval-Augmented Generation (RAG)

Problem: LLMs have stale, generic, parametric knowledge and hallucinate; they can't cite private/changing facts.

Solution: At query time, retrieve relevant passages from an external index (often vector search over embeddings) and inject them into the prompt as grounding context before generation.

Use when: Q&A over private/changing/source-backed documents; you need citations and frequent knowledge updates without retraining.

Avoid when: The need is behavioral/stylistic (tone, format) rather than missing knowledge, or knowledge is static and trivially small; if RAG retrieval proves insufficient, fine-tuning is the next step.

Trade-offs: Gain up-to-date, grounded, citable answers and cheap knowledge updates vs. retraining; pay with retrieval-pipeline complexity (chunking, embeddings, index), added latency, and quality bounded by retrieval relevance.

Related: Data/Model Separation, Online Inference, Guardrails.

Source: AWS Prescriptive Guidance (RAG vs fine-tuning); martinowler.com (GenAI patterns).

Pattern: Online vs Batch Inference

Problem: Different ML use cases have opposite needs - instant per-request answers vs. cheap mass prediction.

Solution: Choose serving mode by freshness need. Online: persistent endpoint, per-request, sub-second latency. Batch: scheduled bulk scoring, results stored/served from cache. Hybrid: precompute offline, lightly re-rank online.

Use when: Online: user-facing, latency-sensitive, unpredictable inputs. Batch: freshness in hours/days, throughput/cost priority (e.g., nightly churn scoring).

Avoid when: Using online for predictable bulk jobs (wasteful) or batch where decisions must be immediate.

Trade-offs: Online gains freshness/immediacy but costs far more per prediction (always-on capacity) and adds latency-engineering complexity; batch gains throughput/cost-efficiency but serves stale predictions.

Related: RAG, Horizontal Scaling, Async Messaging, Cache-Aside.

Source: AWS SageMaker docs (Inference options: real-time vs batch transform); AWS Serverless Lens (Streaming processing).

Pattern: LLM / Model Gateway (Router)

Problem: Many apps calling many model providers directly cause credential sprawl, inconsistent policy, runaway cost, and no central visibility.

Solution: A single gateway in front of all model calls handling routing/fallback (by cost, complexity, availability), centralized auth, rate limiting, logging/observability, and policy enforcement.

Use when: Multiple LLM-powered features and/or providers; you need org-wide governance, cost control, and routing.

Avoid when: A single app calling a single model - the gateway is unnecessary overhead.

Trade-offs: Gain centralized governance, cost/usage visibility, provider flexibility and fallback; pay with an added hop/latency, a potential single point of failure, and an extra component to operate.

Related: API Gateway, Guardrails, RAG, Loose Coupling.

Source: AWS Generative AI Atlas (LLM Gateway); Envoy AI Gateway reference architecture; MLflow AI Gateway guide.

Pattern: AI Guardrails

Problem: LLM inputs/outputs are untrusted and probabilistic - risking PII leakage, prompt injection, off-topic/unsafe content, and malformed outputs.

Solution: Validation/control layers around the model: input checks (intent/relevance, injection detection) and output checks (PII, safety, schema/format). Implemented as LLM-based, embeddings/semantic, or rule-based filters; can run inline at the model gateway.

Use when: User-facing or untrusted-input GenAI, regulated domains, or any output feeding downstream systems needing structure/safety.

Avoid when: Fully trusted, low-risk internal experiments where overhead isn't justified.

Trade-offs: Gain safety, compliance, structured/reliable outputs, defense against injection; pay with added latency, engineering/maintenance cost, and possible false positives blocking valid requests.

Related: Model Gateway, RAG, Security Baseline.

Source: martinfowler.com (GenAI patterns - Guardrails: LLM-based, embeddings, rule-based; NeMo Guardrails); AWS Generative AI Atlas.

Pattern: Loose Coupling

Problem: Tightly coupled components must change and deploy together, so one failure or change ripples across the system.

Solution: Minimize runtime and design-time dependencies - communicate via well-defined interfaces/contracts, async messaging, or events; hide internals behind APIs.

Use when: Virtually always; especially distributed systems, multi-team orgs, and evolving systems.

Avoid when: Never as a principle, but don't over-abstract trivial in-process code where indirection adds needless complexity.

Trade-offs: Gain independent change/deploy/scale, fault isolation, team autonomy; pay with indirection, more moving parts, and (when via async/events) eventual consistency.

Related: Microservices, Event-Driven, API Gateway, Database per Service.

Source: microservices.io (minimize runtime/design-time coupling); AWS Well-Architected; martinfowler.com.

Pattern: Security Baseline - Least Privilege + Encryption

Problem: Over-broad permissions and unprotected data make breaches easy and high-impact.

Solution: Strong identity foundation with least-privilege access (grant only needed permissions, prefer roles/temporary credentials, MFA); encrypt data at rest (e.g., KMS) and in transit (TLS/HTTPS); apply defense in depth across all layers.

Use when: Always - foundational for every workload, doubly so for regulated/sensitive data.

Avoid when: Never; only right-size the rigor to data sensitivity and threat model.

Trade-offs: Gain reduced breach likelihood and blast radius, compliance, auditability; pay with setup/management effort (policy design, key rotation, access reviews) and minor encryption overhead.

Related: API Gateway, Guardrails, Loose Coupling.

Source: AWS Well-Architected Security Pillar (strong identity foundation / least privilege; protect data in transit and at rest); Azure security guidance.